Technical Task: Restaurant Order Sync

Nov 10, 2026 · @momo

Deadline: November 10, 2026, 9:00 PM (Algiers time, UTC+1)

Which task to do

Choose ONE task based on the role you're applying for. You don't need to do both.

- **Frontend / Full-stack roles:** Task A
- **Backend / Systems roles:** Task B

Task A: Frontend (State Management)

Goal: Build a small app that manages the state of a restaurant order and keeps that state consistent for everyone looking at it.

Scenario: An order goes through a lifecycle: **New → Preparing → Ready → Served**. Different people (cashier, kitchen, customer) look at the same order at the same time. When the status changes, everyone sees the new state **instantly**, without refreshing.

What we want to see:

- How you structure the state (what it contains, where it lives)
- How an update reaches every open view
- What happens when two updates happen at the same time

Notes:

- Use any language or framework you want.
- Any approach is fine. What matters is that you can explain why you chose it.
- The UI can be very basic. It does not need to look good.

Task B: Backend

Goal: Build a small system that syncs data between a **cloud database** and a **local database**.

Requirements:

1. Data written locally is synced to the cloud.

2. **If the internet goes down, the app keeps working locally.**
3. When the connection returns, the sync resumes **automatically**, with no manual action.

Notes:

- Use any language, framework or database.
- A simple solution is fine (for example a queue of pending changes). Tell us how you would handle conflicts, even if you don't implement it.
- Don't worry about perfection or edge cases.

What to send us

1. **The code**, as a GitHub link or a zip file.
2. **A short video (3-5 minutes)** where you explain:
    - What you built and how it works
    - Why you chose this approach
    - What you would improve with more time

You can record the video in **any language** you're comfortable with (English, French, Arabic, Darija...).

What we're looking for

This is **not** about building something perfect. We care about:

- **Effort:** you tried and made something that works
- **Understanding:** you know what you built and why
- **Communication:** you can explain your idea clearly

An incomplete solution that is well explained is better than a polished one you can't explain.